

CS561 Research Project: Range Deletes in LSM-Trees

YuCheng Huang
Boston University

MingHan Hsieh
Boston University

Jingyi Li
Boston University

1 BACKGROUND

Log-structured merge (LSM) tree is used extensively in database systems due to its efficiency in writing. To begin with, when the memory is full, data are sorted into a sorted string table (SST) file and flushed onto the disk. Within the disk, the new income file always appends in the back on the first layer. Once data on a layer reach its limitation, files are merged and appended to the next level. And size limitations growth with a ratio, T , from its previous layer. This structure is specialized for inserting and updating the cost of memory and write amplification. Point query and range query are facilitated by bloom filters [4], fence pointers, or range filters [1]. Recently, as described in [3], point delete can also be done with high performance in a timely manner while data correctness is guaranteed. But range delete (RD) so far still costs a tremendous amount of time.

2 PROBLEM STATEMENT

Deleting a range of keys faces several problems. LSM is perfect for the point operation, however when it comes to range operation, it must deal with partially sorted and multiple version data. Although deleting a key can be simply achieved by inserting a tombstone, if the same method is applied for deleting a range of keys, then when a range is large, for example, 1 million, the system is doomed to suffer from high space and write amplification and most of the effort is going to be spent on processing null data. To better tackle the range query, it is better to save the range query as what it is, a range tombstone, within an SST file. As a result, the storage is compact now. To maintain the sorted property, in an SST file, range tombstones are stored in the range section separated from the point data part. Considering that a deleted key may insert again later on, their insert time tags must be kept in the system either implicitly or explicitly in order to know whether a key is currently alive. It is an issue that how we can efficiently maintain such information.

Second, the bloom filter doesn't help with key searching under the range insert. Unlike point insert whose key hash is kept in the bloom filter, the key of range insert isn't kept inside the filter because the bloom filter works well only when the number of inserted keys is relatively sparse. Also inserting all keys within a long range into a bloom filter is inefficient. So IOs are almost unavoidable when checking whether a key is deleted by any range tombstone on each level of an LSM tree.

To deal with that, a solution from RocksDB [2] is to maintain a global delete range information. As the new delete range shall shadow all the delete that arrives beforehand, only the frontier is necessary to be maintained. In this paper, the structure is called the Skyline range delete filter (Skyline RDF). However, constructing and maintaining such information cost extra memory and time. In addition, each search query has to go through the whole system to search for a key first before comparing with the range delete filter (RDF) and is discarded if the data is deleted. Therefore, additional jobs and extra reading efforts are required.

3 OUTLINE

In the following sections. We first introduce the architecture and method we adopted. After briefly introducing our workload, the experiment results are shown. Future work of this project is discussed in the last section.

4 ARCHITECTURE

In this work, each SST is separated into 2 parts, one for point data, the other for saving range tombstone. This project is considered only under the partial leveling LSM system. Instead of maintaining a global range-delete skyline, we propose to keep the RD information on each level and use it as a filter (Perlevel RDF) that stands before bloom filters (BF) as shown in Figure 1. In this way, we prevent extra memory access in the sense that when a query is blocked on the current level, then it doesn't need to search on lower levels. Additionally, our structure eliminates the need of a time tag value recorded in each RD if the system is only required to answer the query based on its latest stored data. This is because, on each level, time information is already embedded in it. When RD moves down to the next level, delete information also moves to the next level. So each RD only needs to record a start value and an end value which saves one-third of the total space compared to the Skyline RDF. However, Perlevel RDF may accumulate the same or overlapped RDs across different levels, which in turn causes space amplification. While on the same level, overlapped RDs are merged into one range which decreases the total required space.

Perlevel RDF are formed when the buffer data is flushed into the disk. Within each SST file, a range delete removes overlapped points inserted earlier than itself but not the points later so when a key is found in a file, then the key truly exists.

When doing compaction, overlapped files across two contiguous levels that is going to be merged were found first. To propagate the delete information down to the next level, range deletes on the current level delete the points on the next level before merging. When merging, point keys merge with point keys, range deletes with range deletes. The exact same thing happened in the Perlevel RDF, ranges come down and merge to the next level. A time tag is not required under this flow because time information is implicitly embedded into each level. When a search query is blocked by Perlevel RDF on the current level, then the searching key wouldn't exist on the level below which prevents unnecessary disk access. This progress is illustrated in Figure 2.

When insert comes down to a level, Perlevel RDF has 2 ways to maintain its ranges as depicted in Figure 3. One is just the same (Perlevel RDF), and another is splitting ranges by the insert point keys (Split_Perlevel RDF). The former one can only block the query from further searching down because it only provides information on whether a key is deleted in the lower level. While in Split_Perlevel RDF, because the key insert information of the current level is written onto the Split_Perlevel RDF of the same level, when a point query is blocked on the current level, then the

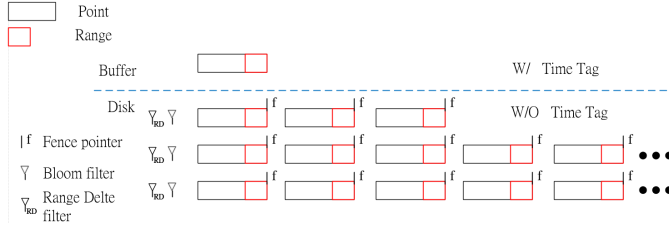


Figure 1: Our LSM Structure with RDF. Comparing to conventional ones, we have RDF deploy on the beginning of each level.

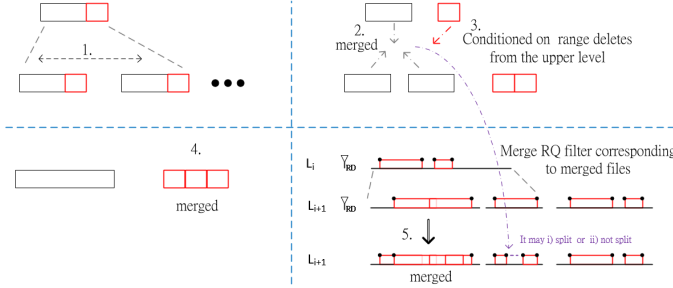


Figure 2: Progress of compaction when Split RDFs exist. On the top left, the system first finds the overlapped files. Then, move to the top right where point keys are merged but those on the lower level are deleted if it falls in some RD ranges of the upper level. Third, on the bottom left, RDs are then merged together. Last, in Split_Perlevel RDF corresponding ranges are merged which is illustrated in the bottom right of the graph.

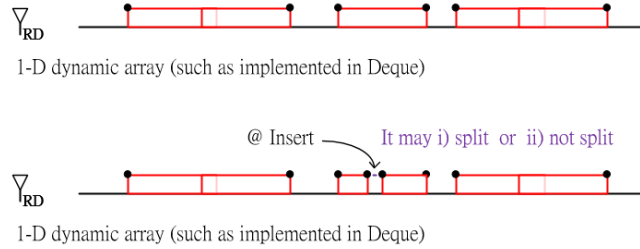


Figure 3: Perlevel RDF can be implemented in two ways either it splits when a later insert falls in its range or not. The top one (Perlevel RDF) is the one that remains the same during key insert. The bottom (Split_Perlevel RDF) shows that each time a new insert comes into its range, it keeps splitting itself so that information on the newly inserted point is kept in the RDF of the same level.

key is guaranteed not to exist on the level. This further decreases the unnecessary disk access which accelerates the searching speed, but at the cost of causing fragmented ranges that can take more memory space than Perlevel RDF one.

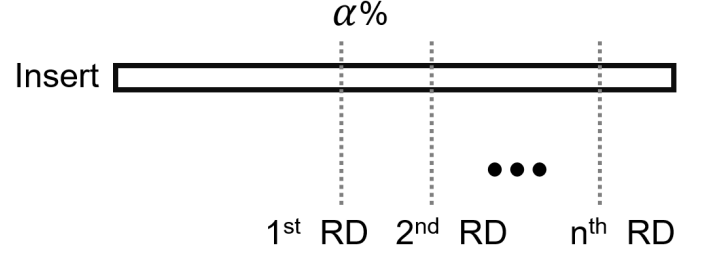


Figure 4: First RD is inserted at the $\alpha\%$ of the total insert, and all the following RDs are equally separated till the end of the insert .

5 WORKLOAD

To test the effect of our implementation, we define our workload. As shown in Figure 4, the 1st RD starts from the α percent of the whole insert workload, and all the following RDs are equally separated to the end of the insert workload. When α moves right, RDs are inserted closer to the end of the input task and have a higher chance to stay on the upper level. In contrast, when α moves left, all the ranges grow older and are more likely to be pushed down to the lower level. Range Delete helps when it stands on the upper level and blocks out unnecessary disk IO access. However, when it goes down to the bottom, the whole system just falls back to the original one without any range delete. In this case, the point query isn't blocked by any RDF, thus the system gains no improvement at all.

6 EXPERIMENT

6.1 Settings

- CPU: Intel Core 17-10750H 2.6GHz
- Total cores: 6
- Total Threads: 12
- RAM: 24 GB

6.2 Results

In Figure 5 and Figure 6, we observe that across all selectivities (0.001, 0.01, 0.1) and different percentages of deleted keys (10%, 50%, 90%), the Perlevel RDF outperforms the Skyline RDF. This is due to the fact that the Perlevel RDF is placed in front of the bloom filter, effectively blocking unnecessary disk IO. This results in a 40% improvement in performance. Furthermore, the Split_Perlevel RDF can further avoid unnecessary disk access.

As shown in Figure 7 and Figure 8, RDs that are fresher and closer to the end of the insert workload block out more unnecessary disk access. This is because ranges staying on the higher levels can effectively block non-existing key queries and prevent them from going deep down to the bottom level.

Although, the improvement on random keys is not as much as the one on deleted keys. There's still an improvement because RDs also contain information about non-existing keys. Therefore, these ranges can help decrease disk IOs and elevate query efficiency.

On the other hand, the current compaction rule prioritizes small overlapped files first and allows ranges in LSM to split by the file

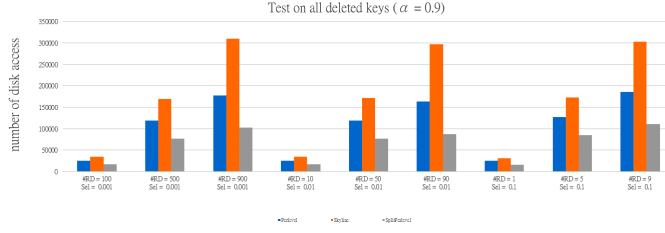


Figure 5: Testing on all deleted keys when $\alpha = 0.9$. From left to right separated into 3 groups [(g1,g2,g3), (g4,g5,g6), (g7,g8,g9)], each of them has selectivity 0.001, 0.01, 0.1 respectively. While the shortest 3 (g1,g4,g7) has 10% of total deleted keys and the middle three (g2,g5,g8) have 50%, whereas the tallest ones (g3,g6,g9) have 90% deleted.

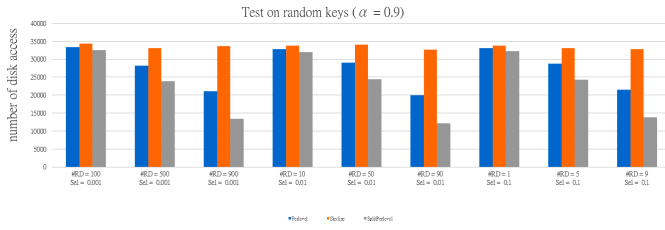


Figure 6: Testing on random keys when $\alpha = 0.9$. Testing on all deleted keys. From left to right separated into 3 groups [(g1,g2,g3), (g4,g5,g6), (g7,g8,g9)], each of them has selectivity 0.001, 0.01, 0.1 respectively. While the shortest 3 (g1,g4,g7) has 10% of total deleted keys and the middle three (g2,g5,g8) have 50%, whereas the tallest ones (g3,g6,g9) have 90% deleted.

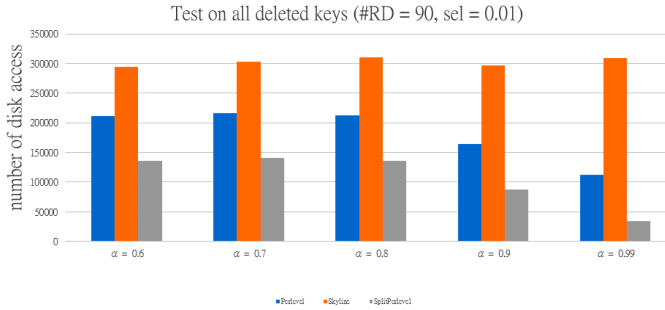


Figure 7: Testing on all deleted keys and varying the head location α of the first inserted RD. All the tasks have a selectivity of 0.01 and have numbers of RDs equal to 90.

boundary in order to fit in the range of a file. However, the compaction rule doesn't consider the completeness of a range, so when only part of a range moves to the next level, a range may be separated into 3 ranges, which causes space amplification. The influence is depicted in Figure 9 and Figure 10. In addition, because insert triggers range split in Split_Perlevel RDF, the space amplification is further exacerbated. When RDs are fresher, ranges stay on the higher level and suffered less from the compaction rule and point insert, thus less space amplification. As long long is used to store

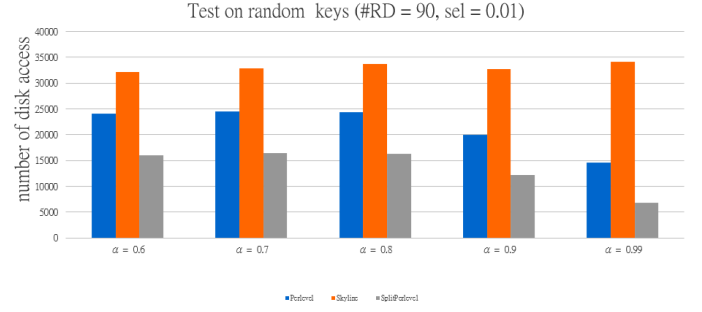


Figure 8: Testing on random keys and varying the head location α of the first inserted RD. All the tasks have a selectivity of 0.01 and have numbers of RDs equal to 90.

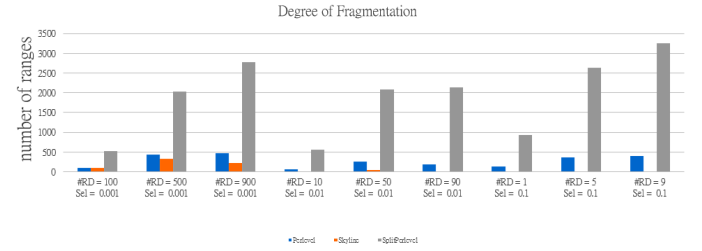


Figure 9: Number of ranges stored in each RDF (Skyline, Perlevel, Split_Perlevel) across 9 different workloads. Ranges in SKyline RDF are represented by triple, while those in Perlevel and Split_Perlevel RDF are 2-value tuple.

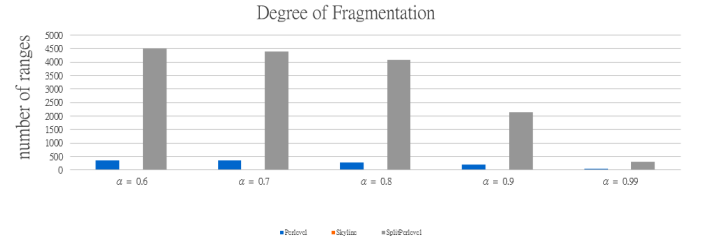


Figure 10: Number of ranges stored in each RDF (Skyline, Perlevel, Split_Perlevel) across different RD freshness. Ranges in SKyline RDF are represented by triple, while those in Perlevel and Split_Perlevel RDF are 2-value tuples.

the range information, Perlevel RDF takes several KB memory footprint, while Split_Perlevel RDF takes tens of KB memory to store ranges for our workload.

7 FUTURE WORK

The current compaction rule leads to space amplification in our RDF. To mitigate this issue, one potential solution is to avoid moving a part of the range to the next level. Instead, we can move the whole ranges down to the next level or in some order so the separated ranges can merge again as soon as possible at the next level. The other thing is that, when RDs grow stale, they take up a significant amount of memory without contributing much to query blocking,

so it is possible to discard a range when it reaches, say, the last 3 levels. Third, as we move to lower levels, the number of files increases, which may result in smaller ranges for each file. We can consider using smaller primitive types to store these ranges to save memory.

REFERENCES

- [1] Siqiang Luo, Subarna Chatterjee, Rafael Ketssetsidis, Niv Dayan, Wilson Qin, and Stratos Idreos. 2020. Rosetta: A Robust Space-Time Optimized Range Filter for Key-Value Stores. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (Portland, OR, USA) (*SIGMOD '20*). Association for Computing Machinery, New York, NY, USA, 2071–2086. <https://doi.org/10.1145/3318464>.
- [2] Abhishek Madan and Andrew Kryczka. 2018. *DeleteRange: A New Native RocksDB Operation*. <https://rocksdb.org/blog/2018/11/21/delete-range.html>
- [3] Subhadeep Sarkar, Tarikul Islam Papon, Dimitris Staratzis, and Manos Athanassoulis. 2020. Lethé: A Tunable Delete-Aware LSM Engine. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (Portland, OR, USA) (*SIGMOD '20*). Association for Computing Machinery, New York, NY, USA, 893–908. <https://doi.org/10.1145/3318464.3389757>
- [4] Zichen Zhu, Ju Hyoung Mun, Aneesh Raman, and Manos Athanassoulis. 2021. Reducing Bloom Filter CPU Overhead in LSM-Trees on Modern Storage Devices. In *Proceedings of the 17th International Workshop on Data Management on New Hardware (DaMoN 2021)* (Virtual Event, China) (*DAMON'21*). Association for Computing Machinery, New York, NY, USA, Article 1, 10 pages. <https://doi.org/10.1145/3465998.3466002>